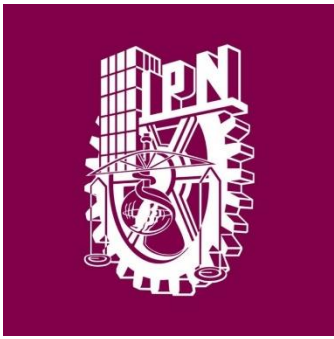




Instituto Politécnico
Nacional
Escuela Superior de
Computo



Profesor: Cortes Galicia Jorge

Materia: Sistemas Operativos

Practica 1

Integrantes:

Vázquez Blancas Cesar Said

Diaz Torres Jonathan Samuel

Aparicio Ponce Roberto Manuel

Corona Hernández Martin Rafael

Introducción Teórica

Los sistemas operativos son una parte fundamental para la correlación entre el usuario y el software, pues los sistemas operativos se encargan de manejar los recursos tanto del software como del hardware, por ellos es que administran las tareas y la administración del tiempo en los procesos, proporcionando una interfaz entre el usuario y el hardware; a pesar de que existen muchos sistemas operativos, de lado de la computadora hay dos muy famosos y conocidos los cuales son Windows y Linux.

Para entender fundamentalmente el concepto de los sistemas operativos es importante entender estas funciones clave:

- Gestión de procesos en función del tiempo.
- Gestión del uso de memoria RAM para asegurar el espacio suficiente.
- Controlar la comunicación entre los dispositivos hardware y software.
- Estructura para almacenamiento y recuperación de datos.
- Interfaz del usuario y su comunicación con el hardware.

Sistema operativo Windows:

Es un sistema operativo muy comercial y de código cerrado, lo que lo hace dominador en el mercado de máquinas de escritorio es que cuenta con una interfaz amigable para todos los usuarios.

Cuenta con:

- **Interfaz gráfica de usuario (GUI):** desde su versión 95 hasta la actualidad su interfaz ha sido mejorada con cada versión mas reciente, esto para facilitar la interacción con el usuario.
- **Sistema de archivos NTFS:** Windows utiliza el sistema de archivos NTFS lo que permite permisos avanzados, compresión, cifrado y soporte para grandes archivos.
- **Soporte extensivo de aplicaciones:** Windows permite instalar la mayoría de las aplicaciones comerciales, lo que hace que estén diseñadas para ejecutarse sin problema en este sistema operativo y poder darles un uso cotidiano.
- **Actualizaciones:** Windows ofrece actualizaciones de seguridad óptimas para cada máquina de escritorio, así como un soporte a largo plazo.

Sistema operativo Linux:

Sistema operativo basado en el kernel de Unix y es de código abierto, es decir que su código esta abierto para su modificación y distribución, lo que lo ha hecho que avance en el terreno de las maquina ser escritorio, sistemas embebidos, servidores de dominio alterno.

Cuenta con:

- **Multitarea y multiusuario:** este sistema operativo este hecho desde sus inicios para soportar múltiples tareas y usuarios.
- **Seguridad:** protege el ingreso a archivos y recursos mediante permisos en grupo y filtros del sistema.
- **Servidores:** Linux es usado para servidores avanzados y sistemas críticos debido a que cuenta con una gran estabilidad, eficiencia, seguridad, robustez y capacidad para mantenerse en funcionamiento durante largos periodos de tiempos y exigencia de procesos.
- **Interfaz de línea de comandos (CLI):** Linux ofrece interfaces gráficas, su potencia y flexibilidad se encuentra en la CLI, lo que hace que permita una personalización y un control avanzados del sistema.

Diferentes tipos de sistemas operativos existen y cada uno tiene su uso particular para diferentes tipos de demandas en cada usuario, por ejemplo:

- **Monousuario:** MS-DOS, Windows 95
- **Multiusuario:** Unix, Linux, Windows Server
- **Tiempo Real (RTOS):** VxWorks, QNX, FreeRTOS
- **Distribuidos:** Plan 9, Apache Hadoop
- **Multiprocesador:** Linux SMP, Windows Server
- **Tiempo Compartido:** Unix, Multics
- **Embebidos:** FreeRTOS, Embedded Linux
- **Red:** Novell NetWare, Windows NT Server
- **Móvil:** Android, iOS
- **De Máquina Virtual:** VMware ESXi, Microsoft Hyper-V

SECCION LINUX:

1. **Reporte la distribución que usted está utilizando. Mencione que otras distribuciones de Linux existen y cuáles son las diferencias respecto a la distribución que usted está utilizando.**

La distribución usada en este entorno es:

Ubuntu

- **Descripción:** Basada en Debian, es una de las distribuciones más populares debido a su facilidad de uso, comunidad activa y gran cantidad de software disponible.
- **Características:** Enfocada en la accesibilidad para usuarios nuevos en Linux, tiene un ciclo de lanzamientos regulares (LTS cada 2 años).
- **Ideal para:** Usuarios principiantes, desarrolladores, servidores web.

Otras distribuciones:

Debian

- **Descripción:** Una distribución muy estable y utilizada como base para muchas otras distribuciones (como Ubuntu).
- **Características:** Prioriza la estabilidad por encima de lo más reciente, lo que la convierte en una excelente opción para servidores.
- **Ideal para:** Usuarios avanzados, servidores, aquellos que priorizan estabilidad.

Fedora

- **Descripción:** Desarrollada y mantenida por la comunidad de Red Hat. Fedora es conocida por estar siempre a la vanguardia en cuanto a nuevas tecnologías.
- **Características:** Uso de software de última generación, ciclos de lanzamientos rápidos.
- **Ideal para:** Desarrolladores, usuarios interesados en las últimas tecnologías de Linux.

Arch Linux

- **Descripción:** Conocida por ser una distribución minimalista y altamente personalizable. No es amigable para principiantes, pero ofrece un control total sobre el sistema.

- **Características:** Modelo de actualización continua (rolling release), filosofía KISS (Keep It Simple, Stupid).
- **Ideal para:** Usuarios avanzados que desean personalizar completamente su sistema.

CentOS

- **Descripción:** Basada en Red Hat Enterprise Linux (RHEL), es ampliamente usada en servidores debido a su estabilidad y largo ciclo de vida.
- **Características:** Distribución estable con un ciclo de lanzamientos largos, ideal para entornos empresariales.
- **Ideal para:** Servidores, entornos de producción.

openSUSE

- **Descripción:** Ofrece dos versiones principales: Leap (estable) y Tumbleweed (rolling release). Es conocida por su herramienta de administración, YaST.
- **Características:** Buena para el uso empresarial y de escritorio, con herramientas avanzadas de administración.
- **Ideal para:** Usuarios que desean estabilidad o un sistema rolling release con buen soporte.

Distribución	Base	Ciclo de Vida	Facilidad	Uso Típico	Destacado por:	Gestor de Paquetes	Seguridad
Ubuntu	Debian	LTS: 5 años (cada 2 años)	Alta	Escritorio, Servidores	Facilidad para principiantes, amplia comunidad	APT (Advanced Packaging Tool)	Alta, con actualizaciones regulares
Debian	Independiente	2-3 años (estable)	Media	Servidores, desarrolladores	Estabilidad y seguridad	APT (Advanced Packaging Tool)	Muy alta, con enfoque en estabilidad
Fedora	Red Hat	1 año por versión	Media	Escritorio, desarrolladores	Software de vanguardia	DNF (Dandified Yum)	Alta, con enfoque en nuevas tecnologías
Arch Linux	Independiente	No definido (rolling release)	Baja	Escritorio (avanzados)	Flexibilidad y personalización	Pacman	Media, depende de la configuración
CentOS	Red Hat	10 años (basado en RHEL)	Media	Servidores	Estabilidad en producción	YUM/DNF	Alta, basado en Red Hat Enterprise
openSUSE	Independiente	Leap: 3-5 años, Tumbleweed: continuo	Media	Escritorio, servidores	Herramienta YaST, versiones Leap y Tumbleweed	Zypper	Alta, estable o rolling release

2. Revise el entorno Linux e identifique los componentes de menú con los que cuenta su sistema operativo. Entre en cada uno de los componentes del menú y explore algunas de sus funcionalidades. Reporte que funcionalidades encontró, además haga un comparativo de estas funcionalidades con respecto a las existentes en el sistema operativo Windows que usted utilice.

Aplicaciones

Este es el menú principal donde están agrupadas todas las aplicaciones instaladas. Se organizan por categorías como Internet, Oficina, Herramientas del Sistema, Sonido y Video, etc.

- **Funcionalidades encontradas:**

- Acceso a todas las aplicaciones instaladas.
- Posibilidad de buscar aplicaciones rápidamente.
- Agrupación de aplicaciones por categorías (Internet, Oficina, Juegos, etc.).
- Posibilidad de anclar aplicaciones favoritas a la barra de tareas

Configuración del sistema (Settings)

Este componente agrupa todas las configuraciones y ajustes del sistema operativo, como la red, sonido, pantalla, energía, etc.

- **Funcionalidades encontradas:**

- Configuración de la red (Wi-Fi, conexiones Ethernet, VPN).
- Ajustes de pantalla (resolución, orientación, pantallas múltiples).
- Configuración de energía (suspensión, brillo, bloqueo de pantalla).
- Ajustes de usuarios (creación y eliminación de cuentas, permisos).
- Administración de dispositivos periféricos (impresoras, ratones, teclados).

Navegador de Archivos (Files o Nautilus)

Este componente permite la gestión de archivos y carpetas en el sistema.

- **Funcionalidades encontradas:**

- Navegación y gestión de archivos (crear, mover, copiar, eliminar carpetas y archivos).
- Integración con discos externos y servidores de red.

- Soporte para pestañas y vistas múltiples (lista o iconos).
- Posibilidad de ejecutar scripts o comandos directamente en directorios.

Terminal (GNOME Terminal)

Herramienta fundamental en sistemas basados en Linux que permite ejecutar comandos y scripts directamente en la línea de comandos.

- **Funcionalidades encontradas:**

- Acceso al sistema mediante comandos.
- Posibilidad de abrir múltiples pestañas o sesiones dentro de una misma ventana.
- Acceso a archivos y administración del sistema sin interfaz gráfica.
- Personalización de temas y atajos de teclado.

Centro de Software (Ubuntu Software)

Es el componente que permite buscar, instalar y gestionar software en el sistema.

- **Funcionalidades encontradas:**

- Instalación de aplicaciones desde un repositorio centralizado.
- Desinstalación y actualización de software.
- Valoraciones y reseñas de usuarios.
- Soporte para aplicaciones Snap y Flatpak.

Centro de Control (System Monitor)

Esta herramienta permite monitorizar el rendimiento del sistema en tiempo real.

- **Funcionalidades encontradas:**

- Monitoreo de uso de CPU, RAM y red.
- Muestra todos los procesos en ejecución con detalles.
- Posibilidad de finalizar procesos.
- Información sobre el rendimiento de la red.

Actualizaciones del sistema (Software Updater)

Herramienta encargada de verificar, descargar e instalar actualizaciones del sistema y software.

- **Funcionalidades encontradas:**

- Verificación automática de actualizaciones.

- Instalación rápida de actualizaciones de seguridad y software.
- Posibilidad de configurar el sistema para que actualice automáticamente.

Redes (Network)

Este menú permite administrar todas las conexiones de red, incluidas Wi-Fi, Ethernet, VPN y Bluetooth.

- **Funcionalidades encontradas:**

- Conexión a redes Wi-Fi y Ethernet.
- Configuración avanzada de redes (direcciones IP estáticas, DNS, proxy).
- Administración de conexiones VPN y configuración de servidores.

Componente	Funcionalidad en Ubuntu	Funcionalidad en Windows	Comparativa
Aplicaciones	Menú de aplicaciones organizado por categorías, búsqueda rápida.	Menú de Inicio, mezcla de accesos directos y programas.	Ubuntu es más limpio, Windows más integrado.
Configuración	Configuración completa de red, usuarios, pantalla, energía, periféricos.	Configuración y Panel de Control.	Ambos son similares, Ubuntu es más personalizable.
Navegador de Archivos	Gestión de archivos, soporte de pestañas y scripts.	Explorador de Archivos	Ubuntu es más potente con herramientas de terminal.
Terminal	GNOME Terminal, acceso avanzado a comandos y scripts.	PowerShell o CMD.	Terminal de Ubuntu es más potente para administración.
Centro de Software	Instalación de software desde repositorios (DEB, Snap, Flatpak).	Microsoft Store.	Ubuntu es más flexible y ofrece más opciones.
Monitor del Sistema	Monitoreo en tiempo real de recursos del sistema.	Administrador de Tareas.	Similares, Ubuntu ofrece más detalles técnicos.
Actualizaciones	Actualización de sistema y software de forma transparente.	Windows Update.	Ubuntu da más control sobre cuándo y qué actualizar.
Redes	Administración avanzada de conexiones Wi-Fi, VPN, Ethernet.	Configuración de Redes e Internet.	Ubuntu ofrece más opciones avanzadas de red.

3. Abra una sesión de consola (Terminal) y reporte para que se utilizan cada uno de los siguientes comandos: para ello utilice el manual en línea (comando man) con el que cuenta Linux. La forma de usarlo es tecleando el siguiente comando en la Terminal: man nombre_del_comando_a_buscar, por ejemplo: man ls, man ps.

ls: Muestra el contenido de un directorio	cd: Cambia el directorio de trabajo actual	cp: Copia archivos o directorios de un lugar a otro
chmod: Cambia los permisos de un archivo o directorio	cat: Muestra el contenido de un archivo en la terminal	mv: Mueve o renombra archivos y directorios
vi: Inicia el editor de texto vi para editar archivos desde la terminal	grep: Busca cadenas de texto en archivos basándose en patrones o expresiones regulares	mkdir: Crea un nuevo directorio
pwd: Muestra el directorio actual en el que te encuentras	rm: Elimina archivos o directorios	rmdir: Elimina directorios vacíos
clear: Limpia la pantalla de la terminal	ps: Muestra una lista de procesos en ejecución	whoami: Muestra el nombre de usuario con el que se está ejecutando la sesión actual

4. Ejecute desde la terminal los siguientes comandos (tenga cuidado al ejecutar los comandos de borrado de archivos y directorios): Reporte los resultados de la ejecución de cada comando.

```
said@said-VirtualBox:~$ ls
Descargas  Escritorio  Música      Público     Vídeos
Documentos Imágenes   Plantillas  snap
```

```
said@said-VirtualBox:~$ ls -l
total 36
drwxr-xr-x 2 said said 4096 sep  1 20:20 Descargas
drwxr-xr-x 2 said said 4096 sep  1 20:20 Documentos
drwxr-xr-x 2 said said 4096 sep  1 20:20 Escritorio
drwxr-xr-x 2 said said 4096 sep  1 20:20 Imágenes
drwxr-xr-x 2 said said 4096 sep  1 20:20 Música
drwxr-xr-x 2 said said 4096 sep  1 20:20 Plantillas
drwxr-xr-x 2 said said 4096 sep  1 20:20 Público
drwx----- 4 said said 4096 sep  1 20:21 snap
drwxr-xr-x 2 said said 4096 sep  1 20:20 Vídeos
```

```
said@said-VirtualBox:~$ ls -la
total 80
drwxr-x--- 16 said said 4096 sep  7 13:53 .
drwxr-xr-x  3 root root 4096 sep  1 20:19 ..
-rw-r--r--  1 said said  220 mar 31 02:41 .bash_logout
-rw-r--r--  1 said said 3771 mar 31 02:41 .bashrc
drwx----- 10 said said 4096 sep  1 20:22 .cache
drwx----- 11 said said 4096 sep  7 13:52 .config
drwxr-xr-x  2 said said 4096 sep  1 20:20 Descargas
drwxr-xr-x  2 said said 4096 sep  1 20:20 Documentos
drwxr-xr-x  2 said said 4096 sep  1 20:20 Escritorio
drwx-----  2 said said 4096 sep  7 13:46 .gnupg
drwxr-xr-x  2 said said 4096 sep  1 20:20 Imágenes
-rw-----  1 said said   20 sep  7 13:53 .lessht
drwx-----  4 said said 4096 sep  1 20:20 .local
drwxr-xr-x  2 said said 4096 sep  1 20:20 Música
drwxr-xr-x  2 said said 4096 sep  1 20:20 Plantillas
-rw-r--r--  1 said said  807 mar 31 02:41 .profile
drwxr-xr-x  2 said said 4096 sep  1 20:20 Público
drwx-----  4 said said 4096 sep  1 20:21 snap
drwx-----  2 said said 4096 sep  1 20:20 .ssh
drwxr-xr-x  2 said said 4096 sep  1 20:20 Vídeos
```

```
said@said-VirtualBox:~$ pwd
/home/said
```

```
said@said-VirtualBox:~$ clear
```

```
said@said-VirtualBox:~$
```

```
said@said-VirtualBox:~$ cd Escritorio/  
said@said-VirtualBox:~/Escritorio$ pwd  
/home/said/Escritorio
```

```
said@said-VirtualBox:~/Escritorio$ ls  
prueba.txt  
said@said-VirtualBox:~/Escritorio$ cat prueba.txt  
hola ubuntu
```

```
said@said-VirtualBox:~/Escritorio$ ls -la|more  
total 12  
drwxr-xr-x  2 said said 4096 sep  7 14:09 .  
drwxr-x--- 16 said said 4096 sep  7 14:08 ..  
-rw-rw-r--  1 said said   12 sep  7 14:09 prueba.txt
```

```
said@said-VirtualBox:~/Escritorio$ ls  
prueba.txt  
said@said-VirtualBox:~/Escritorio$ rm prueba.txt  
said@said-VirtualBox:~/Escritorio$ ls  
said@said-VirtualBox:~/Escritorio$
```

```
said@said-VirtualBox:~/Escritorio$ ps  
  PID TTY          TIME CMD  
 4394 pts/0    00:00:00 bash  
 4448 pts/0    00:00:00 ps
```

```
said@said-VirtualBox:~/Escritorio$ ls  
prueba2.txt  
said@said-VirtualBox:~/Escritorio$ cp prueba2.txt prueba3.txt  
said@said-VirtualBox:~/Escritorio$ ls  
prueba2.txt  prueba3.txt
```

```
said@said-VirtualBox:~/Escritorio$ mv prueba3.txt ../Descargas/  
said@said-VirtualBox:~/Escritorio$ cd ../Descargas/  
said@said-VirtualBox:~/Descargas$ ls  
prueba3.txt
```

```
said@said-VirtualBox:~/Descargas$ mkdir Prueba1  
said@said-VirtualBox:~/Descargas$ ls  
Prueba  Prueba1  prueba3.txt
```

```
said@said-VirtualBox:~/Descargas$ rmdir Prueba
said@said-VirtualBox:~/Descargas$ ls
Prueba1  prueba3.txt
```

```
said@said-VirtualBox:~/Descargas$ whoami
said
```

5.-Algunos de los comandos anteriores pueden tener una serie de opciones para su ejecución (p.e. ls), utilice nuevamente el comando `man nombre_comando` para consultar las diversas opciones que se utilizan en dichos comandos

ls:

- -l: Muestra la lista de archivos en formato largo, incluyendo permisos, propietario, grupo, tamaño y fecha de modificación.
- -a: Incluye archivos ocultos (que comienzan con un punto .).
- -h: Muestra tamaños de archivos en formato legible para humanos (KB, MB, GB) cuando se usa con -l.
- -S: Ordena los archivos por tamaño, de mayor a menor.
- -t: Ordena los archivos por fecha y hora de modificación, mostrando los más recientes primero.
- -r: Invierte el orden de los archivos.
- -R: Muestra el contenido de directorios de manera recursiva, incluyendo subdirectorios.
- --color: Resalta diferentes tipos de archivos con colores para facilitar la identificación visual.
- -i: Muestra el número de inodo de cada archivo.
- -d: Muestra solo los directorios, sin listar su contenido.
- -F: Añade un carácter al final de cada nombre de archivo para indicar el tipo (por ejemplo, / para directorios, * para archivos ejecutables).
- --group-directories-first: Muestra primero los directorios, seguidos de los archivos.
- --ignore=PATTERN: Omite archivos que coincidan con el patrón especificado.
- -U: Lista los archivos en el orden en que aparecen en el directorio, sin ordenar.
- -1: Muestra un archivo por línea, útil para listas largas.

pwd

- **-L:** Muestra la ruta del directorio de trabajo actual tal como está en la variable de entorno PWD, que puede incluir enlaces simbólicos. Si PWD apunta a un directorio simbólico, pwd -L imprimirá la ruta del directorio simbólico seguido de un signo de igualdad y la ruta del directorio de trabajo actual. Esto es útil si deseas ver la ruta simbólica que usaste para llegar al directorio actual.
- **-P:** Muestra la ruta física real del directorio de trabajo actual, resolviendo todos los enlaces simbólicos. Esto significa que el comando pwd -P te dará la ruta exacta en el sistema de archivos sin enlaces simbólicos.

clear

- **clear > archivo.txt:** Limpia la pantalla y redirige la salida vacía a un archivo llamado archivo.txt. Como clear no produce salida, el archivo estará vacío.
- **clear >> archivo.txt:** Añade la salida vacía de clear al final del archivo archivo.txt. Dado que clear no produce salida, el archivo no cambiará.
- El comando clear no produce salida que se pueda redirigir de manera útil, por lo que redirigir su salida generalmente no tiene un efecto práctico.

cd:

- **cd [directorio]:** Cambia al directorio especificado. Puedes usar una ruta absoluta (por ejemplo, /home/usuario/documentos) o relativa (por ejemplo, documentos si estás en /home/usuario).
- **cd ..:** Retrocede un nivel en la jerarquía de directorios, cambiando al directorio padre del directorio actual.
- **cd ~ o cd:** Cambia al directorio de inicio del usuario actual. Usar simplemente cd sin argumentos también te llevará al directorio de inicio.
- **cd -:** Cambia al directorio anterior, es decir, al directorio en el que estabas antes del último cambio de directorio.
- **cd /:** Cambia al directorio raíz del sistema de archivos.

cat:

- **cat [archivo]:** Muestra el contenido del archivo especificado en la salida estándar.
- **cat [archivo1] [archivo2] ...:** Muestra el contenido de múltiples archivos concatenados en la salida estándar, uno tras otro.
- **cat -n [archivo]:** Muestra el contenido del archivo con números de línea en la salida estándar.

- **cat -b [archivo]**: Muestra el contenido del archivo con números de línea, pero solo para líneas no vacías.
- **cat -s [archivo]**: Muestra el contenido del archivo condensando las líneas en blanco consecutivas a una sola línea.
- **cat -E [archivo]**: Muestra el contenido del archivo con un signo de dólar (\$) al final de cada línea, indicando el final de cada línea.
- **cat > [archivo]**: Crea un nuevo archivo con el nombre especificado y permite al usuario escribir contenido en él desde la entrada estándar. Para finalizar la entrada, usa Ctrl + D.
- **cat >> [archivo]**: Añade contenido al final de un archivo existente desde la entrada estándar. Para finalizar la entrada, usa Ctrl + D.
- **cat -A [archivo]**: Muestra el contenido del archivo con caracteres no imprimibles representados de manera visible, como tabulaciones y retornos de carro.

rm:

- **rm [archivo1] [archivo2] ...**: Elimina los archivos especificados en la lista.
- **rm -r [directorio]**: Elimina un directorio y su contenido de forma recursiva, es decir, elimina el directorio junto con todos sus archivos y subdirectorios.
- **rm -f [archivo]**: Elimina el archivo especificado sin pedir confirmación. Esto es útil para evitar solicitudes de confirmación en la eliminación de archivos.
- **rm -i [archivo]**: Solicita confirmación antes de eliminar cada archivo. Es útil para evitar eliminaciones accidentales.
- **rm -v [archivo]**: Muestra información detallada sobre cada archivo que se elimina, proporcionando una visualización de los archivos que están siendo eliminados.
- **rm -d [directorio]**: Elimina solo directorios vacíos. No elimina directorios que contengan archivos o subdirectorios.
- **rm --preserve-root [archivo]**: Previene la eliminación accidental de archivos fuera del directorio raíz (/). Esta opción asegura que no se eliminen archivos o directorios importantes del sistema.
- **rm --help**: Muestra la ayuda del comando rm, proporcionando una lista de todas las opciones disponibles y ejemplos de uso.

ps:

- **ps -e**: Muestra todos los procesos del sistema.
- **ps -u [usuario]**: Muestra los procesos asociados con un usuario específico.
- **ps -aux** o **ps aux**: Muestra todos los procesos del sistema en un formato detallado.
- **ps -f**: Muestra una salida más detallada que incluye información sobre el control de los procesos.
- **ps -l**: Muestra una lista de procesos con información adicional en formato largo.

- **ps -t [terminal]:** Muestra los procesos asociados con una terminal específica.
- **ps -p [PID]:** Muestra información sobre un proceso específico, donde [PID] es el ID de proceso.
- **ps -N:** Muestra solo los procesos cuyos nombres no contienen caracteres ASCII NUL.
- **ps -M:** Muestra la ruta completa del archivo ejecutable en lugar del nombre de comando truncado.

cp:

- **cp [archivo_origen] [archivo_destino]:** Copia el archivo de origen al destino especificado. Si el destino es un directorio, el archivo se copiará en ese directorio con el mismo nombre. Si el destino es un archivo existente, se sobrescribirá.
- **cp -r [directorio_origen] [directorio_destino]:** Copia un directorio y su contenido de forma recursiva al directorio de destino.
- **cp -i [archivo_origen] [archivo_destino]:** Solicita confirmación antes de sobrescribir un archivo existente.
- **cp -v [archivo_origen] [archivo_destino]:** Muestra información detallada sobre los archivos copiados.
- **cp -u [archivo_origen] [archivo_destino]:** Copia solo archivos que son más nuevos que los archivos existentes en el destino o archivos que no existen en el destino.
- **cp -p [archivo_origen] [archivo_destino]:** Conserva los atributos del archivo, como la fecha de modificación y los permisos.
- **cp -a [archivo_origen] [archivo_destino]:** Copia archivos y directorios de forma recursiva y conserva todos los atributos, como los permisos, propietario, grupo, fechas, etc.
- **cp --backup[=CONTROL] [archivo_origen] [archivo_destino]:** Realiza copias de seguridad de los archivos existentes antes de sobrescribirlos. Puedes especificar el tipo de copia de seguridad a través de CONTROL (por ejemplo, --backup=numbered).

mv:

- **mv [archivo_origen] [archivo_destino]:** Mueve el archivo de origen al destino especificado. Si el destino es un directorio, el archivo se moverá a ese directorio. Si el destino es un archivo existente, el archivo de origen se renombrará al nombre de destino.
- **mv -i [archivo_origen] [archivo_destino]:** Solicita confirmación antes de sobrescribir un archivo existente.
- **mv -v [archivo_origen] [archivo_destino]:** Muestra información detallada sobre los archivos movidos o renombrados.

- **mv -u [archivo_origen] [archivo_destino]**: Mueve solo archivos que son más nuevos que los archivos existentes en el destino o archivos que no existen en el destino.
- **mv -f [archivo_origen] [archivo_destino]**: Sobrescribe el archivo de destino sin preguntar, incluso si el archivo de destino es de solo lectura o el archivo de origen no existe.
- **mv -b [archivo_origen] [archivo_destino]**: Realiza copias de seguridad de los archivos existentes antes de sobrescribirlos.
- **mv --backup[=CONTROL] [archivo_origen] [archivo_destino]**: Realiza copias de seguridad de los archivos existentes antes de sobrescribirlos. Puedes especificar el tipo de copia de seguridad a través de CONTROL (por ejemplo, --backup=numbered).

mkdir:

- **mkdir [directorio]**: Crea un directorio con el nombre especificado en el directorio actual.
- **mkdir -p [directorio]**: Crea un directorio y, si es necesario, también crea cualquier directorio padre que falte en la ruta. Esta opción es útil para crear una estructura de directorios jerárquica.
- **mkdir -m [modo] [directorio]**: Especifica los permisos del nuevo directorio mediante el modo octal (por ejemplo, mkdir -m 755 carpeta).
- **mkdir -v [directorio]**: Muestra información detallada sobre los directorios creados.
- **mkdir --help**: Muestra la ayuda del comando mkdir, proporcionando una lista de opciones y ejemplos de uso.

rmdir:

- **rmdir [directorio]**: Elimina el directorio especificado si está vacío. Si el directorio no está vacío, rmdir mostrará un mensaje de error.
- **rmdir -p [directorio]**: Elimina el directorio especificado y, de forma recursiva, también elimina cualquier directorio padre vacío que quede después de eliminar el directorio especificado. Esta opción es útil para eliminar una estructura de directorios vacíos de forma eficiente.
- **rmdir --ignore-fail-on-non-empty [directorio]**: Elimina el directorio especificado incluso si no está vacío. Ten en cuenta que esta opción puede causar la eliminación de archivos y subdirectorios no deseados si el directorio no está vacío.
- **rmdir --help**: Muestra la ayuda del comando rmdir, proporcionando una lista de opciones y ejemplos de uso.

whoami:

Muestra el nombre del usuario asociado con el proceso que está ejecutando el comando. Es un comando simple sin parámetros ni opciones adicionales.

6.- Abra el editor de texto de su preferencia y realice un programa en lenguaje C que imprima en pantalla “Hola mundo”. Compile y ejecute su programa desde la Terminal, para ello utilice el comando siguiente: gcc nombre_programa.c -o nombre_salida para compilar, y ./nombre_salida para ejecutar el programa.

```
#include <stdio.h>

int main(){
    printf("Hola mundo");
    return 0;
}
```

```
said@said-VirtualBox:~/Documentos$ gcc holaMundo.c -o holaMundo
said@said-VirtualBox:~/Documentos$ ./holaMundo
Hola Mundosaid@said-VirtualBox:~/Documentos$
```

7.- Realice un programa que muestre la siguiente salida:

```
      *      *
    ***      *
  *****      *
*****
      *      *****
      *      ***
      *      *
```

El dato de entrada del programa será el número de asteriscos de la línea horizontal en la figura con la mayor cantidad de ellos (por ejemplo: para la figura dada es 14). Este dato se pedirá al usuario y podrá ser cualquier valor dentro de un rango que usted establezca, el cual permita visualizar correctamente la figura en la pantalla. A partir de este único dato, se construirá la figura con la restricción de que sólo

deberá imprimirse un asterisco a la vez. Adicionalmente, dé la opción de guardar la figura en un archivo. Utilice sólo bibliotecas estándares de ANSI C. Reporte tanto el código fuente como las pantallas de ejecución del programa.

```
#include<stdio.h>
void imprimirFigura(int n){
    int e,a,i,j;
    for(i=0;i<n;i++){
        e = n - i -1;
        a = 2 * i + 1;
        for(j=0;j<e;j++){
            printf(" ");
        }
        if (e > 0){
            for(j=0;j<a;j++){
                printf("*");
            }
        }
        for(j=0;j<e+ (n - 1);j++){
            printf(" ");
        }
    }
    else{
        for(j=0;j<a*2;j++){
            printf("*");
        }
    }
    printf("\n");
}
for(i=n-1;i>0;i--){
    e = n - i;
    a = 2 * i - 1;
    for(j=0;j<n-1;j++){
        printf(" ");
    }
    printf("*");
    for(j=0;j<n-1+e;j++){
        printf(" ");
    }
}
```

```

        for(j=0;j<a;j++){
            printf("*");
        }
        printf("\n");
    }
}

int main(){
    int n=0,b,i;
    printf("Ingrese el numero de niveles de su piramide: ");
    scanf("%d",&b);
    if (b % 2 == 0)
    {
        b++;
    }

    for (i = 0; i < (b / 2); i++)
    {
        b -= 2;

        if (b>= 0)
        {
            n++;
        }
    }

    imprimirFigura(n);
    return 0;
}

```

Ingrese el numero de niveles de su piramide: 14

```

    *      *
   ***    *
  ***** *
 *****
 *      *****
 *      ***
 *      *
```

8.-Realice un programa que resuelva las torres de Hanoi para n discos de forma recursiva. Reporte tanto el código fuente como las pantallas de ejecución del programa.

```
#include <stdio.h>

void hanoi(int n, char origen, char destino, char auxiliar) {
    if (n == 1) {
        printf("Mover disco 1 de %c a %c\n", origen, destino);
        return;
    }
    hanoi(n - 1, origen, auxiliar, destino);
    printf("Mover disco %d de %c a %c\n", n, origen, destino);

    hanoi(n - 1, auxiliar, destino, origen);
}

int main() {
    int n;

    printf("Ingresa el número de discos: ");
    scanf("%d", &n);
    hanoi(n, 'A', 'C', 'B');

    return 0;
}
```

```
said@said-VirtualBox:~/Documentos$ gcc hanoi.c -o hanoi
said@said-VirtualBox:~/Documentos$ ./hanoi
Ingresa el número de discos: 3
Mover disco 1 de A a C
Mover disco 2 de A a B
Mover disco 1 de C a B
Mover disco 3 de A a C
Mover disco 1 de B a A
Mover disco 2 de B a C
Mover disco 1 de A a C
```

9.-Realice un programa que determine si un grupo de paréntesis están balanceados utilizando pilas. Deberán considerarse tres tipos de paréntesis: (), { }, []. Reporte tanto el código fuente como las pantallas de ejecución del programa.

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct Node
{
    char data;
    struct Node* next;
} Node;
typedef struct Stack
{
    Node* top;
} Stack;

void init(Stack* stack){
    stack->top=NULL;
}

int isEmpty(Stack* stack){
    if(stack->top==NULL){
        return 1;
    }else{
        return 0;
    }
}

void push(Stack* stack, char item){
    Node* node = (Node*)malloc(sizeof(Node));
    node->data=item;
    node->next=stack->top;
    stack->top=node;
}
```

```
char pop(Stack* stack){
    if(isEmpty(stack)){
        printf("La pila esta vacia");
        return -1;
    }
    Node* node = stack->top;
    char item= stack->top->data;
    stack->top= node->next;
    free(node);
    return item;
}

int peek(Stack* stack){
    if(isEmpty(stack)){
        printf("La pila esta vacia");
        return -1;
    }
    return stack->top->data;
}

void imprimir(Stack* stack){
    if(isEmpty(stack)){
        printf("La pila esta vacia");
    }else{
        Node* node=stack->top;
        while(node!=NULL){
            printf("%d",node->data);
            node = node->next;
        }
    }
}
```

```

int balancear(Stack* stack, char *cadena) {
    int i;
    char e;
    for (i = 0; cadena[i] != '\0'; i++) {
        if (cadena[i] == '(' || cadena[i] == '{' || cadena[i] == '[') {
            push(stack, cadena[i]);
        } else if (cadena[i] == ')' || cadena[i] == '}' || cadena[i] == ']') {
            if (isEmpty(stack)) {
                return 0;
            }
            e = pop(stack);
            if ((cadena[i] == ')' && e != '(') ||
                (cadena[i] == ']' && e != '[') ||
                (cadena[i] == '}' && e != '{')) {
                return 0;
            }
        }
    }
    return isEmpty(stack);
}

```

```

int main(){
    Stack stack;
    char cadena[100];
    init(&stack);
    printf("Ingrese una expresion con parentesis, corchetes o llaves: ");
    fflush(stdout);
    if (fgets(cadena, sizeof(cadena), stdin) == NULL)
    {
        printf("Error al leer cadena.\n");
        return 1;
    }
    cadena[strcspn(cadena, "\n")] = '\0';

    if(balancear(&stack,cadena)){
        printf("estan balanceados");
    }else{
        printf("esta desbalanceados");
    }
}

```

```

said@said-VirtualBox:~/Documentos$ gcc balancear.c -o balan
said@said-VirtualBox:~/Documentos$ ./balan
Ingrese una expresion con parentesis, corchetes o llaves: (){((()){})(
estan balanceadossaid@said-VirtualBox:~/Documentos$ ./balan
Ingrese una expresion con parentesis, corchetes o llaves: ({}
esta desbalanceadossaid@said-VirtualBox:~/Documentos$

```

10. Realice un programa que evalúe cualquier expresión aritmética delimitada completamente con paréntesis balanceados (por ejemplo, entrada: $((5-2)+4)*10$), salida expresión evaluada: 70). Como restricción del programa se tiene que no deberá de utilizar notación prefija ni posfija para la evaluación. Reporte tanto el código fuente como las pantallas de ejecución del programa.

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
typedef struct Node {
    double value;
    struct Node* next;
} Node;

typedef struct Stack {
    Node* top;
} Stack;

void init(Stack* stack) {
    stack->top = NULL;
}

int isEmpty(Stack* stack) {
    return stack->top == NULL;
}

void push(Stack* stack, double value) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->value = value;
    node->next = stack->top;
    stack->top = node;
}
```

```
double pop(Stack* stack) {
    if (isEmpty(stack)) {
        printf("Error: pila vacia\n");
        exit(1);
    }
    Node* node = stack->top;
    double value = node->value;
    stack->top = node->next;
    free(node);
    return value;
}

double evaluate(char operator, double operand1, double operand2) {
    switch (operator) {
        case '+': return operand1 + operand2;
        case '-': return operand1 - operand2;
        case '*': return operand1 * operand2;
        case '/':
            if (operand2 == 0) {
                printf("Error: división por cero\n");
                exit(1);
            }
            return operand1 / operand2;
        default:
            printf("Error: operador no válido\n");
            exit(1);
    }
}
```

```

int balancear(Stack* stack, char *cadena) {
    int i;
    char e;
    for (i = 0; cadena[i] != '\0'; i++) {
        if (cadena[i] == '(' || cadena[i] == '{' || cadena[i] == '[') {
            push(stack, cadena[i]);
        } else if (cadena[i] == ')' || cadena[i] == '}' || cadena[i] == ']') {
            if (isEmpty(stack)) {
                return 0;
            }
            e = pop(stack);
            if ((cadena[i] == ')' && e != '(') ||
                (cadena[i] == ']' && e != '[') ||
                (cadena[i] == '}' && e != '{')) {
                return 0;
            }
        }
    }
    return isEmpty(stack);
}

```

```

double evaluateExpression(const char *cadena) {
    Stack stack;
    init(&stack);
    char operador = 0;
    double operando = 0;
    int i = 0;
    if(balancear(&stack,cadena)){
        while (cadena[i] != '\0') {
            if (isdigit(cadena[i])) {
                operando = 0;
                while (isdigit( cadena[i])) {
                    operando = (operando * 10) + (cadena[i] - '0');
                    i++;
                }
                push(&stack, operando);
            } else if (cadena[i] == '+' || cadena[i] == '-' ||
                cadena[i] == '*' || cadena[i] == '/') {
                operador = cadena[i];
                i++;
            } else if (cadena[i] == ')') {
                double operand2 = pop(&stack);
                double operand1 = pop(&stack);
                double result = evaluate(operador, operand1, operand2);
                push(&stack, result);
                i++;
            } else {
                i++;
            }
        }
    }else{
        return -1;
    }
    return pop(&stack);
}

```



```

int main() {
    char expression[100];
    printf("Ingrese una expresión aritmética con paréntesis balanceados: ");
    fflush(stdout);
    if (fgets(expression, sizeof(expression), stdin) == NULL) {
        printf("Error al leer la expresión.\n");
        return 1;
    }
    expression[strcspn(expression, "\n")] = '\0';

    double resultado = evaluateExpression(expression);
    if(resultado!=-1){
        printf("El resultado de la expresión es: %lf\n", resultado);
    }else{
        printf("Esta desbalanceado");
    }

    return 0;
}

```

```

said@said-VirtualBox:~/Documentos$ ./exp
Ingrese una expresión aritmética con paréntesis balanceados: (((5-2)+4)*10)
El resultado de la expresión es: 70.000000
said@said-VirtualBox:~/Documentos$ ./exp
Ingrese una expresión aritmética con paréntesis balanceados: ((4-3)(*6)
Esta desbalanceadosaid@said-VirtualBox:~/Documentos$

```

11. Guarde todos sus programas (fuente y ejecutable) en una memoria usb.

SECCION WINDOWS

1. Inicie sesión en Windows.
2. Abra una consola.
3. Investigue en Internet para que sirve cada uno de los siguientes comandos y ejecútelos en la consola (tenga cuidado al ejecutar los comandos de borrado de archivos y directorios).

Dir	Cd nombre_directorio	Copy nombre_archivo nuevo_nombre_archivo
ipconfig	Type nombre_archivo	Ren nombre_archivo nuevo_nombre_archivo
Cls	Mkdir nombre_directorio	Chdir nombre_directorio
Ver	rmdir nombre_directorio	echo "Hola mundo"
Tree	del nombre_archivo	Find "cadena_buscar" nombre_archivo

Reporte la función de cada uno de los anteriores comandos ejecutados.

Dir: Muestra una lista de archivos y directorios en el directorio actual o en un directorio especificado. Este comando permite ver los archivos y carpetas que se encuentran en una ubicación específica del sistema de archivos.

- /p: Pausa la salida después de cada pantalla llena de información.
- /w: Muestra la lista en formato de ancho (solo nombres de archivo).
- /s: Muestra archivos en el directorio actual y en todos los subdirectorios.
- /b: Muestra solo los nombres de archivo, sin detalles adicionales.
- /o: Ordena los archivos. Puedes especificar n (por nombre), e (por extensión), d (por fecha), etc.

```
C:\Users\vbcsl>dir
El volumen de la unidad C es Windows-SSD
El número de serie del volumen es: C6A9-0150

Directorio de C:\Users\vbcsl

01/09/2024  14:02    <DIR>        .
08/06/2024  00:38    <DIR>        ..
24/07/2024  19:25                175 .angular-config.json
07/06/2024  20:30    <DIR>        .cache
30/07/2024  20:44    <DIR>        .m2
08/06/2024  17:40    <DIR>        .matplotlib
19/06/2024  00:52    <DIR>        .nbi
06/09/2024  12:45                176 .packettracer
07/06/2024  20:48    <DIR>        .quokka
07/09/2024  15:07    <DIR>        .VirtualBox
07/06/2024  20:25    <DIR>        .vscode
07/06/2024  20:43    <DIR>        .wallaby
06/09/2024  12:46    <DIR>        Cisco Packet Tracer 8.2.2
07/06/2024  19:08    <DIR>        Contacts
07/06/2024  19:11    <DIR>        Documents
07/09/2024  14:52    <DIR>        Downloads
07/06/2024  19:08    <DIR>        Favorites
20/08/2024  16:48    <DIR>        frontend-tareas
01/07/2024  22:57    <DIR>        GlassFish_Server
07/06/2024  19:08    <DIR>        Links
07/06/2024  19:08    <DIR>        Music
07/09/2024  12:24    <DIR>        OneDrive
11/08/2024  19:18    <DIR>        PycharmProjects
07/06/2024  19:08    <DIR>        Saved Games
07/06/2024  21:40    <DIR>        Searches
12/06/2024  17:10    <DIR>        Videos
01/09/2024  19:59    <DIR>        VirtualBox VMs
                2 archivos                351 bytes
                25 dirs 385.007.456.256 bytes libres
```

Cd: Cambia el directorio de trabajo actual a uno nuevo. El comando cd es fundamental para la navegación en la línea de comandos de Windows, permitiendo moverse entre directorios.

- cd ..: Regresa al directorio padre del directorio actual.
- cd \: Cambia al directorio raíz del disco actual.
- cd /d [unidad]:\[directorio]: Cambia a un directorio en una unidad diferente.

```
C:\Users\vbcs1> cd ..  
C:\Users>|
```

Copy: Copia uno o más archivos de un lugar a otro. Este comando es útil para duplicar archivos o moverlos a diferentes ubicaciones manteniendo el archivo original intacto.

- /a: Copia archivos en modo texto.
- /b: Copia archivos en modo binario.
- /y: Suprime la solicitud de confirmación antes de sobrescribir archivos existentes.

```
c:\Users\vbcs1>copy prueba.txt prueba2.txt  
1 archivo(s) copiado(s).
```

Ipconfig: Muestra la configuración de red del sistema, incluyendo la dirección IP, máscara de subred y puerta de enlace predeterminada para cada interfaz de red.

- /all: Muestra información detallada sobre todas las interfaces de red, incluyendo información sobre servidores DNS, direcciones MAC y más.
- /release: Libera la dirección IP obtenida por DHCP.
- /renew: Renueva la dirección IP obtenida por DHCP.

```

c:\Users\vbcs1>ipconfig

Configuración IP de Windows

Adaptador de Ethernet Ethernet 2:

    Sufijo DNS específico para la conexión. . . :
    Vínculo: dirección IPv6 local. . . . . : fe80::65a8:ebed:3c9c:ea60%38
    Dirección IPv4. . . . . : 192.168.56.1
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . . :

Adaptador de LAN inalámbrica Conexión de área local* 1:

    Estado de los medios. . . . . : medios desconectados
    Sufijo DNS específico para la conexión. . . :

Adaptador de LAN inalámbrica Conexión de área local* 2:

    Estado de los medios. . . . . : medios desconectados
    Sufijo DNS específico para la conexión. . . :

Adaptador de LAN inalámbrica Wi-Fi:

    Sufijo DNS específico para la conexión. . . :
    Dirección IPv6 . . . . . : 2806:105e:24:260a:be56:af18:701d:f66c
    Dirección IPv6 temporal. . . . . : 2806:105e:24:260a:a04d:dcbl:d87b:3f3e
    Vínculo: dirección IPv6 local. . . . . : fe80::9ae4:978e:f3b:5818%14
    Dirección IPv4. . . . . : 192.168.1.78
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . . : fe80::1%14
                                                192.168.1.254

Adaptador de Ethernet Conexión de red Bluetooth:

    Estado de los medios. . . . . : medios desconectados
    Sufijo DNS específico para la conexión. . . :

c:\Users\vbcs1>

```

Type: Muestra el contenido de un archivo en la pantalla. Este comando se utiliza para ver el contenido de archivos de texto.

No tiene opciones adicionales. Se utiliza para visualizar el contenido de archivos de texto.

```

c:\Users\vbcs1>type prueba.txt
Hola Windows
c:\Users\vbcs1>type prueba2.txt
Hola Windows

```

Ren: Renombra uno o más archivos o directorios. Este comando permite cambiar el nombre de un archivo o carpeta en el sistema.

No tiene opciones adicionales. Utiliza este comando para renombrar el archivo o directorio especificado.

```
c:\Users\vbcsl>dir
El volumen de la unidad C es Windows-SSD
El número de serie del volumen es: C6A9-0150

Directorio de c:\Users\vbcsl

07/09/2024 16:19 <DIR>      .
08/06/2024 00:38 <DIR>      ..
24/07/2024 19:25          175 .angular-config.json
07/06/2024 20:30 <DIR>      .cache
30/07/2024 20:44 <DIR>      .m2
08/06/2024 17:40 <DIR>      .matplotlib
19/06/2024 00:52 <DIR>      .nbi
06/09/2024 12:45          176 .packettracer
07/06/2024 20:48 <DIR>      .quokka
07/09/2024 16:18 <DIR>      .VirtualBox
07/06/2024 20:25 <DIR>      .vscode
07/06/2024 20:43 <DIR>      .wallaby
06/09/2024 12:46 <DIR>      Cisco Packet Tracer 8.2.2
07/06/2024 19:08 <DIR>      Contacts
07/06/2024 19:11 <DIR>      Documents
07/09/2024 14:52 <DIR>      Downloads
07/06/2024 19:08 <DIR>      Favorites
20/08/2024 16:48 <DIR>      frontend-tareas
01/07/2024 22:57 <DIR>      GlassFish_Server
07/06/2024 19:08 <DIR>      Links
07/06/2024 19:08 <DIR>      Music
07/09/2024 12:24 <DIR>      OneDrive
07/09/2024 16:09          12 prueba.txt
11/08/2024 19:18 <DIR>      PycharmProjects
07/06/2024 19:08 <DIR>      Saved Games
07/06/2024 21:40 <DIR>      Searches
07/09/2024 16:09          12 test.txt
12/06/2024 17:10 <DIR>      Videos
01/09/2024 19:59 <DIR>      VirtualBox VMs
                4 archivos          375 bytes
                25 dirs 388.212.895.744 bytes libres

c:\Users\vbcsl>ren test.txt RENOMBRADO.txt

c:\Users\vbcsl>dir
El volumen de la unidad C es Windows-SSD
El número de serie del volumen es: C6A9-0150

Directorio de c:\Users\vbcsl

07/09/2024 16:20 <DIR>      .
08/06/2024 00:38 <DIR>      ..
24/07/2024 19:25          175 .angular-config.json
07/06/2024 20:30 <DIR>      .cache
30/07/2024 20:44 <DIR>      .m2
08/06/2024 17:40 <DIR>      .matplotlib
19/06/2024 00:52 <DIR>      .nbi
06/09/2024 12:45          176 .packettracer
07/06/2024 20:48 <DIR>      .quokka
07/09/2024 16:18 <DIR>      .VirtualBox
07/06/2024 20:25 <DIR>      .vscode
07/06/2024 20:43 <DIR>      .wallaby
06/09/2024 12:46 <DIR>      Cisco Packet Tracer 8.2.2
07/06/2024 19:08 <DIR>      Contacts
07/06/2024 19:11 <DIR>      Documents
07/09/2024 14:52 <DIR>      Downloads
07/06/2024 19:08 <DIR>      Favorites
20/08/2024 16:48 <DIR>      frontend-tareas
01/07/2024 22:57 <DIR>      GlassFish_Server
07/06/2024 19:08 <DIR>      Links
07/06/2024 19:08 <DIR>      Music
07/09/2024 12:24 <DIR>      OneDrive
07/09/2024 16:09          12 prueba.txt
11/08/2024 19:18 <DIR>      PycharmProjects
07/09/2024 16:09          12 RENOMBRADO.txt
07/06/2024 19:08 <DIR>      Saved Games
07/06/2024 21:40 <DIR>      Searches
12/06/2024 17:10 <DIR>      Videos
```

Cls: Limpia la pantalla de la ventana de comandos. Esto elimina todo el texto de la pantalla y deja la línea de comandos limpia para nuevas entradas.

No tiene opciones adicionales. Solo limpia la pantalla de la ventana de comandos.

```
Directorio de c:\Users\vbcs1

07/09/2024  16:20    <DIR>          .
08/06/2024  00:38    <DIR>          ..
24/07/2024  19:25                175 .angular-config.json
07/06/2024  20:30    <DIR>          .cache
30/07/2024  20:44    <DIR>          .m2
08/06/2024  17:40    <DIR>          .matplotlib
19/06/2024  00:52    <DIR>          .nbi
06/09/2024  12:45                176 .packettracer
07/06/2024  20:48    <DIR>          .quokka
07/09/2024  16:18    <DIR>          .VirtualBox
07/06/2024  20:25    <DIR>          .vscode
07/06/2024  20:43    <DIR>          .wallaby
06/09/2024  12:46    <DIR>          Cisco Packet Tracer 8.2.2
07/06/2024  19:08    <DIR>          Contacts
07/06/2024  19:11    <DIR>          Documents
07/09/2024  14:52    <DIR>          Downloads
07/06/2024  19:08    <DIR>          Favorites
20/08/2024  16:48    <DIR>          frontend-tareas
01/07/2024  22:57    <DIR>          GlassFish_Server
07/06/2024  19:08    <DIR>          Links
07/06/2024  19:08    <DIR>          Music
07/09/2024  12:24    <DIR>          OneDrive
07/09/2024  16:09                12 prueba.txt
11/08/2024  19:18    <DIR>          PycharmProjects
07/09/2024  16:09                12 RENOMBRADO.txt
07/06/2024  19:08    <DIR>          Saved Games
07/06/2024  21:40    <DIR>          Searches
12/06/2024  17:10    <DIR>          Videos
01/09/2024  19:59    <DIR>          VirtualBox VMs
                4 archivos                375 bytes
                25 dirs 388.212.723.712 bytes libres

c:\Users\vbcs1>cls
```

```
c:\Users\vbcs1>
```

Mkdir: Crea un nuevo directorio con el nombre especificado en la ubicación actual. Este comando es fundamental para la organización de archivos en directorios.

No tiene opciones adicionales. Solo crea el directorio especificado.

```
c:\Users\vbcs1>mkdir Prueba
```

```
07/09/2024 17:51 <DIR> Prueba
```

Chdir: Cambia el directorio de trabajo actual al especificado. chdir es una forma alternativa de cd, y se utiliza para cambiar la ubicación en la línea de comandos.

- chdir ..: Regresa al directorio padre.
- chdir \: Cambia al directorio raíz.

```
c:\Users\vbcs1>chdir Prueba
```

```
c:\Users\vbcs1\Prueba>|
```

Ver: Este comando generalmente se utiliza para mostrar la versión de un comando o el sistema operativo, no para ver archivos. La documentación de Windows no menciona ver como un comando para ver contenido de archivos.

No tiene opciones adicionales para mostrar archivos.

```
c:\Users\vbcs1\Prueba>ver
```

```
Microsoft Windows [Versión 10.0.22631.4037]
```

Rmdir: Elimina un directorio vacío. Para eliminar un directorio que contiene archivos o subdirectorios, debes usar la opción /s con el comando rd.

- /s: Elimina el directorio y todos sus archivos y subdirectorios.
- /q: Suprime la confirmación al eliminar directorios no vacíos.

```
c:\Users\vbcs1\Prueba>chdir ..
```

```
c:\Users\vbcs1>rmdir Prueba
```

```
c:\Users\vbcs1>chdir Prueba
```

```
El sistema no puede encontrar la ruta especificada.
```

Echo: Imprime el texto especificado en la salida estándar (la pantalla). Este comando es útil para mostrar mensajes, realizar pruebas o redirigir texto a archivos.

- echo on: Activa el eco de los comandos ejecutados.
- echo off: Desactiva el eco de los comandos.

```
c:\Users\vbcs1>echo "Hola Windows"
"Hola Windows"
```

Tree: Muestra una representación gráfica en forma de árbol de los directorios y subdirectorios en el directorio actual. Este comando facilita la visualización de la estructura de directorios.

- /f: Muestra los archivos en cada directorio.
- /a: Usa caracteres ASCII en lugar de caracteres extendidos para mostrar el árbol.

```
├── packaging-23.2.dist-info
├── dist
├── l10n
├── resources
├── ms-python.debugpy-2024.8.0-win32-x64
│   ├── bundled
│   │   └── libs
│   │       ├── debugpy
│   │       │   ├── adapter
│   │       │   ├── common
│   │       │   ├── launcher
│   │       │   ├── server
│   │       │   └── vendored
│   │       │       └── pydevd
│   │       │           ├── pydevd_attach_to_process
│   │       │           ├── common
│   │       │           ├── linux_and_mac
│   │       │           ├── winappdbg
│   │       │           │   └── win32
│   │       │           └── windows
│   │       │               ├── pydevd_plugins
│   │       │               │   ├── extensions
│   │       │               │   └── types
│   │       │               ├── pydev_ipython
│   │       │               ├── pydev_sitecustomize
│   │       │               ├── _pydevd_bundle
│   │       │               │   ├── pydevd_concurrency_analyser
│   │       │               │   ├── _debug_adapter
│   │       │               ├── _pydevd_frame_eval
│   │       │               └── vendored
│   │       │                   ├── bytecode
│   │       │                   │   └── tests
│   │       │                   └── bytecode-0.13.0.dev0.dist-info
│   │       │                       ├── _pydev_bundle
│   │       │                       ├── fsnotify
│   │       │                       └── _pydev_runfiles
│   │       ├── debugpy-1.8.2.dist-info
│   │       ├── packaging
│   │       └── packaging-23.2.dist-info
│   ├── dist
│   ├── l10n
│   └── resources
│   └── ms-python.python-2024.12.3-win32-x64
│       ├── dist
│       ├── images
│       ├── l10n
│       ├── languages
│       ├── out
│       │   └── client
│       │       ├── node_modules
│       │       └── unicode
│       │           └── category
│       ├── python-env-tools
│       │   ├── bin
│       │   └── docs
```


Del: Elimina uno o más archivos del sistema. Este comando se usa para borrar archivos de manera permanente.

- /q: Suprime la confirmación al eliminar archivos.
- /f: Fuerza la eliminación de archivos de solo lectura.

```
c:\Users\vbcs1>dir
El volumen de la unidad C es Windows-SSD
El número de serie del volumen es: C6A9-0150

Directorio de c:\Users\vbcs1

07/09/2024  18:04    <DIR>          .
08/06/2024  00:38    <DIR>          ..
24/07/2024  19:25                175 .angular-config.json
07/06/2024  20:30    <DIR>          .cache
30/07/2024  20:44    <DIR>          .m2
08/06/2024  17:40    <DIR>          .matplotlib
19/06/2024  00:52    <DIR>          .nbi
06/09/2024  12:45                176 .packettracer
07/06/2024  20:48    <DIR>          .quokka
07/09/2024  16:18    <DIR>          .VirtualBox
07/06/2024  20:25    <DIR>          .vscode
07/06/2024  20:43    <DIR>          .wallaby
06/09/2024  12:46    <DIR>          Cisco Packet Tracer 8.2.2
07/06/2024  19:08    <DIR>          Contacts
07/06/2024  19:11    <DIR>          Documents
07/09/2024  14:52    <DIR>          Downloads
07/06/2024  19:08    <DIR>          Favorites
20/08/2024  16:48    <DIR>          frontend-tareas
01/07/2024  22:57    <DIR>          GlassFish_Server
07/06/2024  19:08    <DIR>          Links
07/06/2024  19:08    <DIR>          Music
07/09/2024  17:48    <DIR>          OneDrive
11/08/2024  19:18    <DIR>          PycharmProjects
07/09/2024  16:09                12 RENOMBRADO.txt
07/06/2024  19:08    <DIR>          Saved Games
07/06/2024  21:40    <DIR>          Searches
12/06/2024  17:10    <DIR>          Videos
01/09/2024  19:59    <DIR>          VirtualBox VMs

           3 archivos             363 bytes
          25 dirs 388.425.170.944 bytes Libres

c:\Users\vbcs1>
```

Find: Busca una cadena de texto específica dentro de un archivo y muestra las líneas que contienen esa cadena. Este comando es útil para localizar texto en archivos.

- /i: Ignora las diferencias entre mayúsculas y minúsculas durante la búsqueda.
- /c: Muestra solo el recuento de las líneas que contienen la cadena buscada.

```
c:\Users\vbcs1>find "Hola" RENOMBRADO.txt

----- RENOMBRADO.TXT
Hola Windows
```

4.-Conecte su memoria usb donde tiene los programas que desarrolló en la sección de Linux y ejecútelos directamente. Reporte sus observaciones sobre la ejecución de los mismos.

```
c:\Users\vbcs1\Ubuntu>./holaMundo
"." no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.
```

No logra ejecutar las salidas compiladas en Linux (Ubuntu). Windows no reconoce el archivo

5.-Busque el directorio donde está instalado Dev C o algún IDE de C que tenga como compilador a gcc.

6. Desde la consola ubíquese en el directorio de Dev C, y cambie al directorio "bin".

7. Desde ese directorio compile en la consola como se indica en el punto 6 de la sección Linux cada uno de los programas creados en esa sección y que guardó en la memoria usb. Reporte la ejecución de todos los programas.

```
c:\Users\vbcs1\C>gcc holaMundo.c -o holaMundo

c:\Users\vbcs1\C>holaMundo
Hola mundo
```

```
C:\Users\vbcs1\C>gcc figura.c -o figura

C:\Users\vbcs1\C>figura
Ingresa el numero de niveles de su piramide: 14
  *      *
 ***     *
*****   *
*****
  *      *****
  *      ***
  *      *
```

```
c:\Users\vbcs1\C>gcc hanoi.c -o hanoi
```

```
c:\Users\vbcs1\C>hanoi
```

```
Ingresa el número de discos: 3
```

```
Mover disco 1 de A a C
```

```
Mover disco 2 de A a B
```

```
Mover disco 1 de C a B
```

```
Mover disco 3 de A a C
```

```
Mover disco 1 de B a A
```

```
Mover disco 2 de B a C
```

```
Mover disco 1 de A a C
```

```
c:\Users\vbcs1\C>gcc balanceo.c -o bal
```

```
c:\Users\vbcs1\C>bal
```

```
Ingresa una expresion con parentesis, corchetes o llaves: ((())){[(())]}
```

```
están balanceados
```

```
c:\Users\vbcs1\C>bal
```

```
Ingresa una expresion con parentesis, corchetes o llaves: (({[[]])
```

```
esta desbalanceada
```

```
c:\Users\vbcs1\C>exp
```

```
Ingresa una expresi|n aritm|tica con par|ntesis balanceados: (((5-2)+4)*10)
```

```
El resultado de la expresi|n es: 70.000000
```

```
c:\Users\vbcs1\C>exp
```

```
Ingresa una expresi|n aritm|tica con par|ntesis balanceados: (4*7(8+9)
```

```
Esta desbalanceado
```

8. Reporte las diferencias y similitudes entre los comandos de Linux y Windows, así como la compilación y ejecución de los programas realizados, que haya encontrado.

Diferencias:

1. Sistema Operativo y Distribuciones

- **Linux:** Es un sistema operativo de código abierto basado en Unix. Existen múltiples distribuciones (distros) de Linux, como Ubuntu, Fedora, CentOS, Debian, entre otras, cada una diseñada para satisfacer diferentes necesidades. Estas distribuciones pueden variar en cuanto a herramientas preinstaladas, gestión de paquetes y personalización.
- **Windows:** Es un sistema operativo comercial desarrollado por Microsoft. Las versiones más comunes son Windows 7, 8, 10, y 11, con cada iteración ofreciendo mejoras en la interfaz de usuario, seguridad y rendimiento.

Windows es de código cerrado, lo que limita las modificaciones que los usuarios pueden hacer al sistema.

2. Entorno de Línea de Comandos

- **Linux:** Utiliza la Terminal, que es un entorno de línea de comandos potente y flexible. La Terminal de Linux está basada en el shell, con bash siendo el más común, aunque también se pueden utilizar otros como zsh o fish. Los usuarios interactúan con el sistema mediante comandos de texto, y es posible automatizar tareas a través de scripts.
- **Windows:** Cuenta con el Símbolo del sistema (Command Prompt) y PowerShell. PowerShell es más avanzado que el Símbolo del sistema tradicional, ya que permite el uso de comandos basados en .NET, proporcionando una mayor capacidad de administración del sistema. Aunque ambas herramientas son poderosas, el entorno de comandos de Linux es generalmente considerado más flexible y rico en comandos nativos.

3. Comandos del Sistema

- **Linux:** Los comandos de Linux son generalmente basados en Unix y pueden diferir mucho de los de Windows. Por ejemplo, para listar archivos y directorios se usa ls, para cambiar directorios se utiliza cd, y para eliminar archivos se usa rm. Linux permite una mayor manipulación del sistema de archivos y los permisos a través de la línea de comandos.
- **Windows:** Los comandos en Windows varían del entorno de Linux. El comando dir se utiliza para listar archivos, y cd para cambiar de directorio (igual que en Linux, pero con sintaxis levemente diferente). Windows depende en gran medida de su interfaz gráfica para la mayoría de las tareas de administración, aunque el uso de PowerShell ha incrementado la capacidad para realizar tareas complejas mediante scripting.

4. Compilación y Ejecución de Programas en C

- **Linux:** Utiliza GCC (GNU Compiler Collection) como el compilador estándar para lenguajes como C y C++. Para compilar un programa en C, se ejecuta el comando gcc nombre_archivo.c -o nombre_programa. Una vez compilado, el programa se ejecuta con ./nombre_programa. Linux facilita la automatización de tareas de compilación con herramientas como make.
- **Windows:** En Windows se pueden utilizar entornos de desarrollo como Dev C++, Code::Blocks, o Visual Studio, que integran compiladores como GCC o MSVC. Los programas en C se compilan y generan archivos ejecutables (.exe), que se pueden ejecutar directamente. Aunque Windows también

soporta GCC, la instalación y configuración suelen ser más complejas que en Linux.

5. Estructura de Directorios

- **Linux:** Sigue una estructura de directorios jerárquica con el directorio raíz representado como /. Los directorios principales incluyen /home (donde se encuentran los archivos de usuario), /etc (configuraciones del sistema), /var (archivos variables como logs), y /usr (aplicaciones de usuario). Todo en Linux es tratado como un archivo, incluidas las unidades de hardware, que se montan en puntos dentro de este sistema de archivos.
- **Windows:** Utiliza una estructura de unidades con letras (como C:\, D:\) para identificar discos y particiones. El directorio raíz de la unidad del sistema es C:\, y los directorios clave incluyen C:\Program Files para las aplicaciones, C:\Users para los perfiles de usuario, y C:\Windows para los archivos del sistema operativo. A diferencia de Linux, las unidades de hardware no están integradas directamente en la estructura de archivos.

6. Gestión de Archivos y Permisos

- **Linux:** Utiliza un sistema avanzado de permisos basado en usuarios y grupos, con tres niveles de permisos: lectura, escritura y ejecución, que pueden aplicarse a archivos y directorios. Los usuarios pueden modificar los permisos mediante los comandos chmod y chown. Además, Linux permite sistemas de archivos como ext4, Btrfs, y XFS, con capacidades avanzadas como instantáneas y control de versiones.
- **Windows:** Windows también tiene un sistema de permisos, aunque es más orientado a la interfaz gráfica. Los permisos en Windows se manejan a través del Explorador de archivos o con herramientas como el Administrador de permisos. Los sistemas de archivos más comunes son NTFS y exFAT, con NTFS proporcionando características como permisos avanzados, cifrado y control de cuotas.

7. Seguridad

- **Linux:** Es considerado más seguro en general, debido a su sistema de permisos y la comunidad activa que corrige vulnerabilidades rápidamente. Los virus y malware son menos comunes, aunque no inexistentes.
- **Windows:** Históricamente ha sido más vulnerable a virus y malware debido a su popularidad y algunas deficiencias de seguridad en versiones anteriores. Sin embargo, Windows 10 y 11 han mejorado significativamente en términos de seguridad, con características como Windows Defender y la actualización constante de parches de seguridad.

Similitudes:

1. Comandos del Sistema

Aunque los comandos específicos en Linux y Windows son diferentes, muchas de las funcionalidades básicas que ofrecen son similares:

- **Listar archivos y directorios:** Ambos sistemas permiten visualizar el contenido de directorios mediante comandos similares (ls en Linux y dir en Windows). Aunque varíen en nombre y opciones, la función principal es la misma.
- **Cambiar de directorio:** El comando cd se utiliza tanto en Linux como en Windows para moverse entre directorios, lo que proporciona una experiencia común para la navegación en línea de comandos.
- **Copiar archivos:** Ambas plataformas permiten copiar archivos de un lugar a otro con comandos similares, como cp en Linux y copy en Windows. La sintaxis y opciones varían, pero la funcionalidad se mantiene equivalente.

2. Compilación y Ejecución de Programas en C

Ambos sistemas operativos pueden utilizar el compilador **GCC (GNU Compiler Collection)** para la compilación de programas en C y otros lenguajes. Esto ofrece una experiencia similar en términos de desarrollo de software:

- **Proceso de compilación:** En ambos sistemas, el proceso de compilación sigue un flujo parecido: se utiliza gcc nombre_archivo.c -o nombre_programa para generar el archivo ejecutable. Este flujo es consistente tanto en Linux como en Windows, aunque el entorno de compilación puede diferir.
- **Ejecución de programas:** Una vez compilado, el programa se ejecuta desde la línea de comandos, aunque con ligeras variaciones en la sintaxis. En Linux se usa ./nombre_programa, mientras que en Windows se ejecuta como nombre_programa.exe, pero ambos procesos son conceptualmente idénticos.

3. Capacidad para Ejecutar Programas Compilados

En ambos sistemas, es posible ejecutar programas compilados desde la línea de comandos, una vez que se han cumplido los requisitos de permisos:

- **Permisos:** En **Linux**, el archivo debe tener permisos de ejecución, que se pueden configurar con el comando chmod +x. En **Windows**, los programas .exe pueden ejecutarse directamente si los permisos de seguridad del sistema lo permiten. Ambos sistemas requieren configuraciones específicas de permisos para la ejecución de software, pero el principio de otorgar permisos es común.

4. Uso de Pilas y Evaluación de Expresiones

Ambos sistemas permiten el uso de estructuras de datos como **pilas** y la evaluación de expresiones, aunque se realicen a través de herramientas o lenguajes diferentes:

- **Evaluación de expresiones:** Tanto en Linux como en Windows se pueden escribir scripts o programas que evalúen expresiones matemáticas o realicen tareas como verificar el balanceo de paréntesis. Linux suele utilizar lenguajes de scripting como **bash** para esto, mientras que Windows puede utilizar **PowerShell** o incluso lenguajes como **Batch** o **C#** para resolver estos problemas.
- **Pilas:** Aunque no es un comando del sistema, ambos sistemas pueden implementar estructuras de datos como pilas en aplicaciones o scripts, demostrando que los conceptos de programación subyacentes son compartidos.

5. Almacenamiento y Ejecución de Programas desde Unidades Externas

En ambos sistemas, los programas y archivos se pueden almacenar en dispositivos de almacenamiento externo como memorias USB o discos duros externos:

- **Acceso y ejecución desde unidades externas:** Tanto Linux como Windows permiten la ejecución de programas y la apertura de archivos directamente desde dispositivos externos. En Linux, las unidades suelen montarse en puntos de montaje específicos como `/media`, mientras que en Windows se accede a ellas a través de letras de unidad como `E:\`. Una vez montadas o asignadas, los programas y archivos pueden ejecutarse o leerse de manera similar en ambos sistemas.

6. Acceso a Internet y Búsqueda de Información

Tanto Linux como Windows permiten el acceso a Internet para buscar información, descargar actualizaciones, o realizar tareas relacionadas con el desarrollo de software:

- **Navegadores web y herramientas:** Ambos sistemas ofrecen navegadores web como **Firefox**, **Chrome**, o **Edge**, permitiendo buscar información sobre comandos y soluciones a problemas. Además, en ambos sistemas se pueden instalar aplicaciones de línea de comandos como **curl** o **wget** para realizar solicitudes HTTP directamente desde el terminal o consola.
- **Acceso a recursos de desarrollo:** Linux y Windows ofrecen acceso a plataformas de desarrollo y repositorios de software como **GitHub** o **Bitbucket**, permitiendo a los desarrolladores clonar repositorios y colaborar en proyectos desde ambos sistemas operativos.

7. Automatización y Scripting

Tanto Linux como Windows tienen potentes capacidades de automatización y scripting:

- **Scripting:** En Linux, se suele utilizar **bash** o **sh** para crear scripts que automatizan tareas del sistema. En Windows, **PowerShell** proporciona una herramienta similar para crear scripts complejos con acceso a la API del sistema operativo. Ambos sistemas permiten crear scripts para realizar tareas repetitivas o configurar entornos.
- **Automatización:** Las tareas administrativas y de mantenimiento en ambos sistemas pueden automatizarse. Por ejemplo, es posible programar tareas en Linux mediante **cron**, mientras que en Windows se utiliza el **Programador de Tareas**.

8. Gestión de Paquetes y Software

Si bien el enfoque de cada sistema es diferente, ambos permiten instalar y gestionar software fácilmente:

- **Linux:** Utiliza gestores de paquetes como **apt**, **dnf**, o **yum** para instalar, actualizar y eliminar software desde repositorios centralizados. Esto permite a los usuarios gestionar todo su software de forma eficiente.
- **Windows:** Utiliza herramientas como **Microsoft Store**, **winget**, o **Chocolatey** para gestionar software. Estos gestores facilitan la instalación de aplicaciones, especialmente en los entornos de desarrollo, de una forma que se asemeja a los gestores de paquetes de Linux.

Conclusión

Esta práctica nos permitió como equipo introducirnos ya no solo de manera teórica sino también práctica al uso y conocimiento de ¿qué es? y ¿cómo se usa? Un sistema operativo. También nos ayudó a volver a ocupar un lenguaje de programación distinto a otros que más utilizamos a diario como es el lenguaje C, así también reconocer otro tipo de distribuciones de Linux y su gran gamma de comandos para poder usar de forma mas eficiente este sistema operativo, sus principales características y diferenciaciones entre lo que es un sistema operativo de código cerrado (Windows) y otro de código abierto como puede ser Linux, en fin, una practica interesante en este inicio de curso y bastante importante para introducirnos en este rubro del Hardware y del Software.